

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5. Preprocessing

Perform the following preprocessing steps:

1. Load the raw inertial signals.
2. Arrange every observation window as 128×9 .
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

Suggested split:

70% training, 15% validation, 15% testing.

The test data must not be used during model selection or hyperparameter tuning.

Expected Preprocessing Output

Students must print:

Input tensor shape:

```
Training   : (N_train, 128, 9)
Validation : (N_val,   128, 9)
Testing    : (N_test,  128, 9)
```

Number of classes: 6
Number of features per time step: 9
Sequence length: 128

The actual values of N_{train} , N_{val} and N_{test} depend on the selected subset.

6. Temporal Data Visualization

Representative sensor sequences were selected from three activity classes: WALKING, WALKING UPSTAIRS, and SITTING. The plots show how the sensor values change over 128 time steps for three channels: body_acc_x, body_acc_y, and body_gyro_x.

Plot 1: Sensor signal versus time

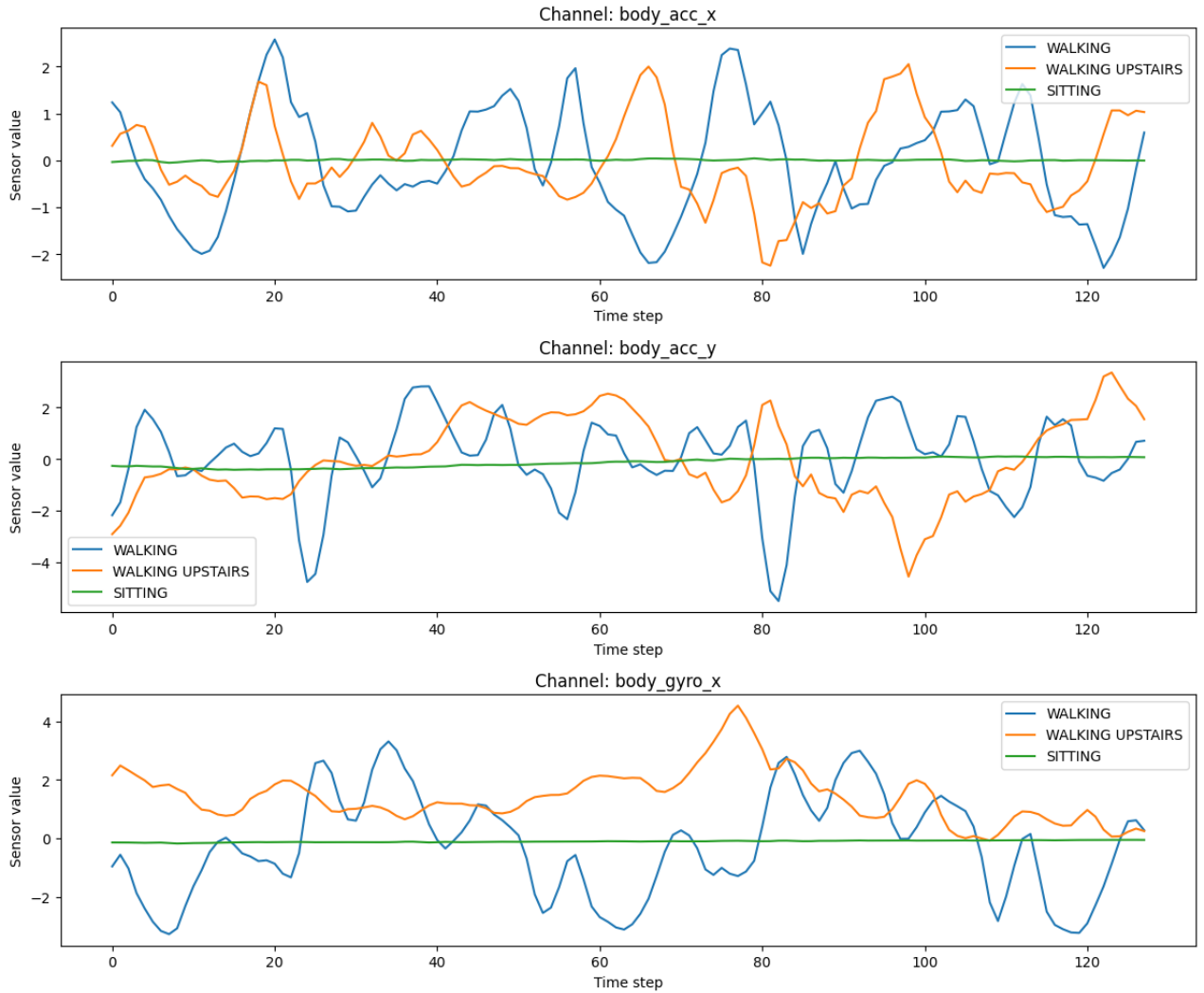


Figure 1: Temporal sensor signals for WALKING, WALKING UPSTAIRS, and SITTING

Inference:

1. What temporal pattern is visible?

WALKING and WALKING UPSTAIRS show clear variations and repeated oscillations in the sensor values over time. The WALKING signal changes between positive and negative values several times, while WALKING UPSTAIRS also shows continuous variations with some larger peaks. In comparison, the SITTING signal remains almost flat and close to zero in all three channels, showing much less movement.

2. Which activities exhibit similar patterns?

WALKING and WALKING UPSTAIRS show more similar temporal behaviour because both involve continuous movement and have repeated changes in acceleration and gyroscope values. SITTING is different from these two activities because its sensor values remain relatively stable. This difference also explains why some walking-related activities can be confused with each other during classification.

3. Why is the ordering of measurements important?

The order of sensor measurements is important because an activity is represented not only by the individual sensor values but also by how those values change with time. For example, the repeated increases, decreases, and peaks in WALKING and WALKING UPSTAIRS represent movement patterns. If the same measurements were randomly reordered, this temporal pattern would be lost, making it harder for a recurrent model to identify the activity correctly.

7. Vanilla RNN Architecture

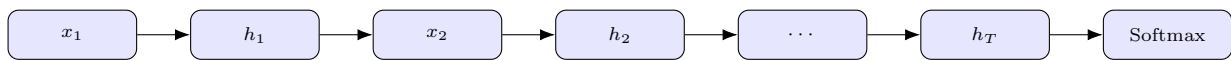
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

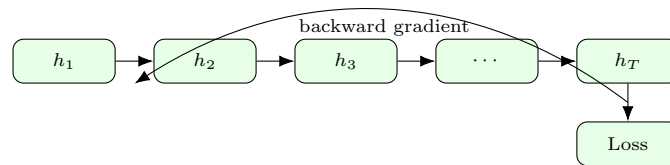
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Students should explain that repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Consider

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2,$$

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

Using

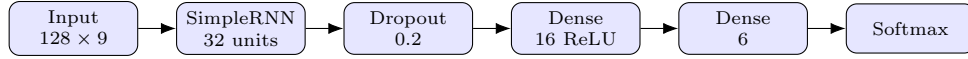
$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

calculate h_1 , h_2 and h_3 manually.

Students must compare their calculated values with the values obtained from a program.

9. RNN Implementation

Use a small recurrent architecture:



Suggested training configuration:

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

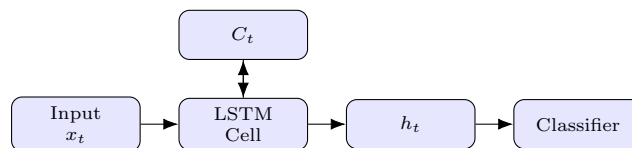
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



Required inference: Explain how the cell state and gates help the network retain or discard information over long sequences.

11. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

and

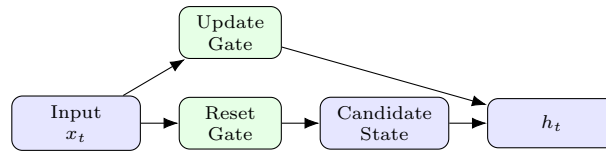
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$



Students should compare the number of gates and state representations in RNN, LSTM and GRU.

12. LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

All other experimental conditions should remain unchanged.

13. Required Training Plots

The RNN, LSTM and GRU models all were trained for 30 epochs using same training settings. The training and validation loss and accuracy curves were plotted for all three models.

RNN

RNN Training and Validation Loss

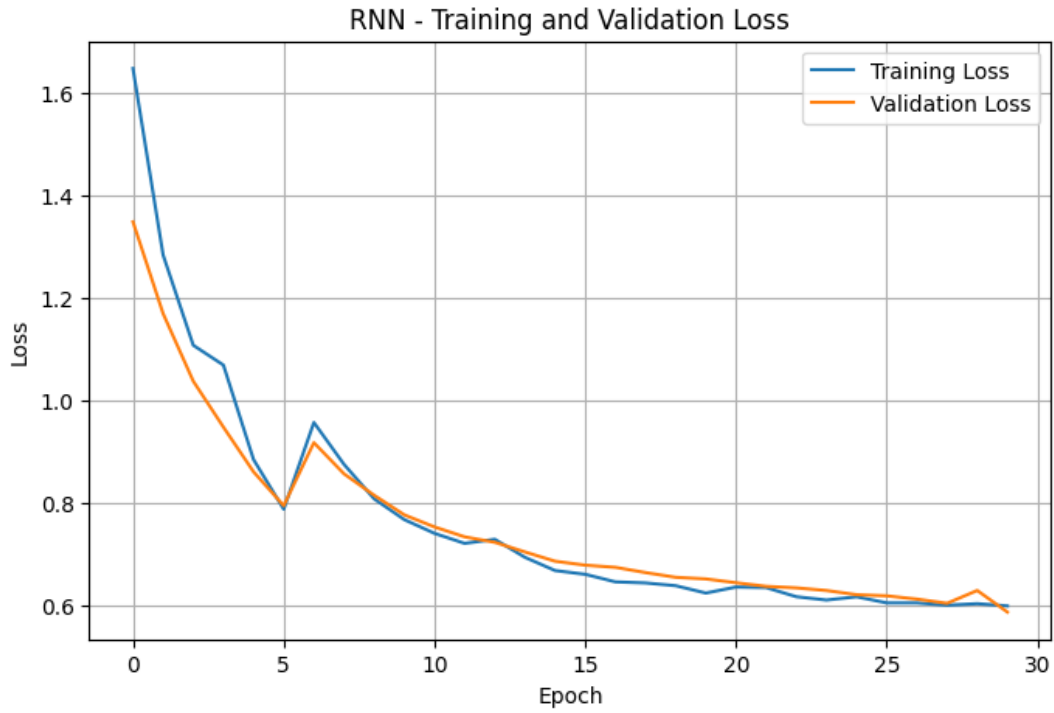


Figure 2: RNN Training and Validation Loss

Inference:

The RNN loss decreases from the initial high value and reached about 0.60 by end of the training. The training and validation losses stay quite close to each other, so there is no large generalization gap between them. However, the loss does not become very small and the accuracy remains around 74%, which is showing that the RNN is still underfitting this task.

RNN Training and Validation Accuracy

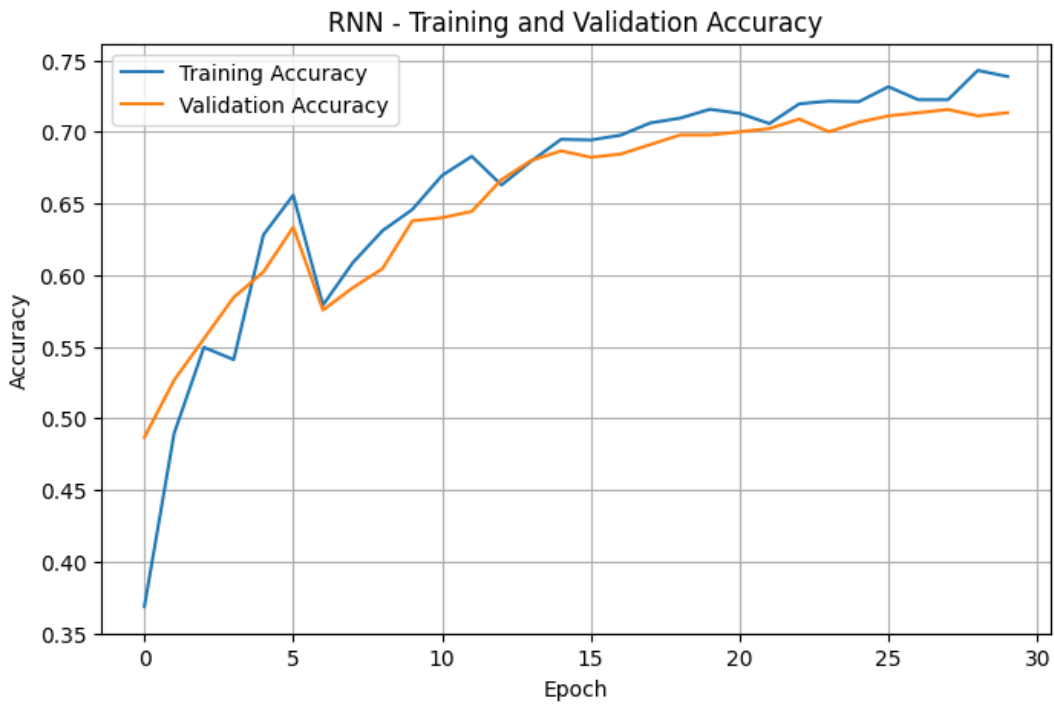


Figure 3: RNN Training and Validation Accuracy

Inference:

The RNN accuracy improves slowly from about 37% training accuracy in the first epoch to 73.86% by epoch 30. The validation accuracy also increased from 48.67% to 71.33%. The curves remain fairly close, but both stop at a much

lower accuracy itself than the LSTM and GRU, showing that the simple RNN did not learn the activity patterns as well.

LSTM

LSTM Training and Validation Loss

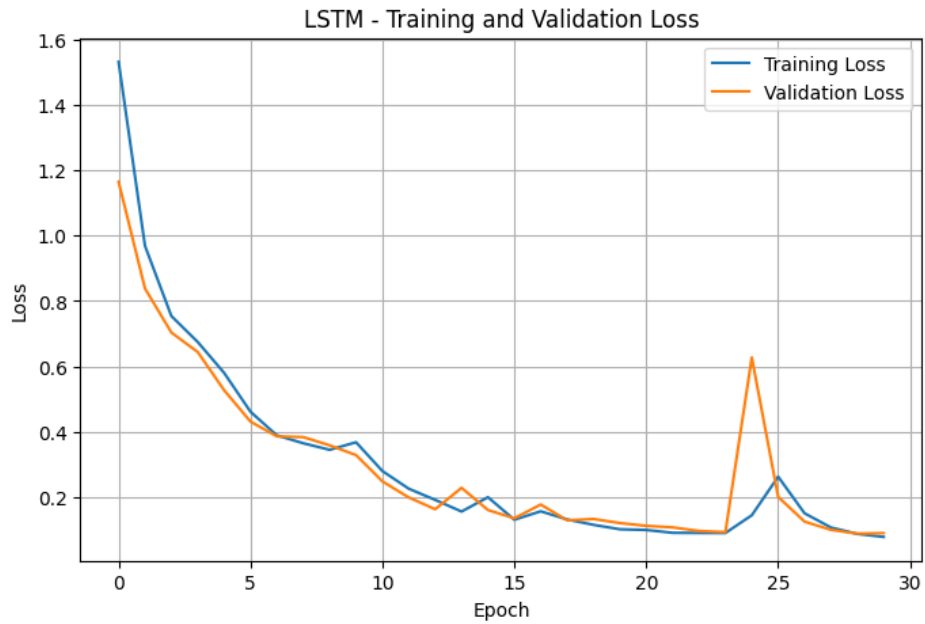


Figure 4: LSTM Training and Validation Loss

Inference:

The LSTM loss falls quickly during training and reached 0.0783 for training and 0.0899 for validation at epoch 30. There is one clear temporary increase in validation loss around epoch 25, but it comes down again in the next epochs. The final losses are close, showing that the LSTM learned the training patterns well while still giving similar validation performance.

LSTM Training and Validation Accuracy

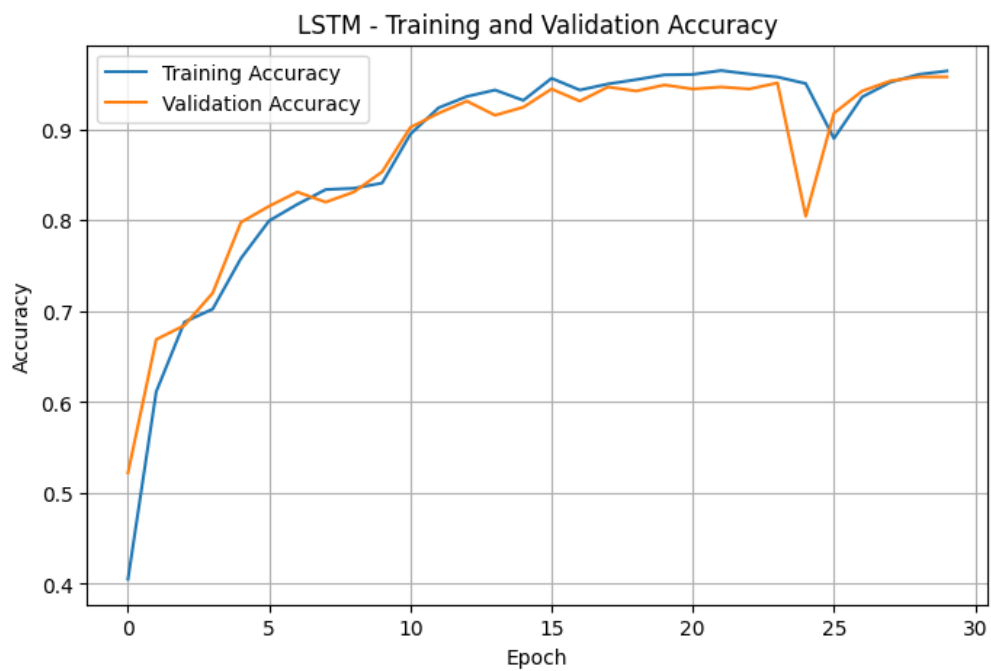


Figure 5: LSTM Training and Validation Accuracy

Inference:

The LSTM accuracy increases from about 40.52% training accuracy to 96.43% by the final epoch. Validation accuracy also increases to 95.78%. There is a sudden validation drop around epoch 25, but it recovers in the following epochs. The small final gap between training and validation accuracy shows good generalization in this experiment.

GRU

GRU Training and Validation Loss

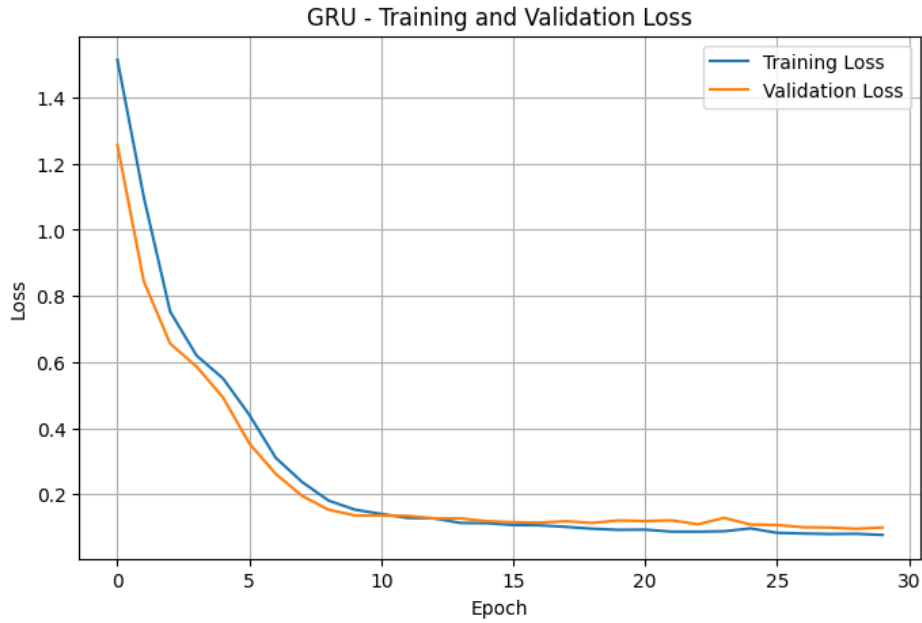


Figure 6: GRU Training and Validation Loss

Inference:

The GRU loss decreased quickly from about 1.52 to below 0.1 during our training. After around epoch 10, both losses became much more stable and remain close to each other. At epoch 30 the training loss is 0.0758 and validation loss is 0.0862, showing that the model has converged with only a small gap.

GRU Training and Validation Accuracy

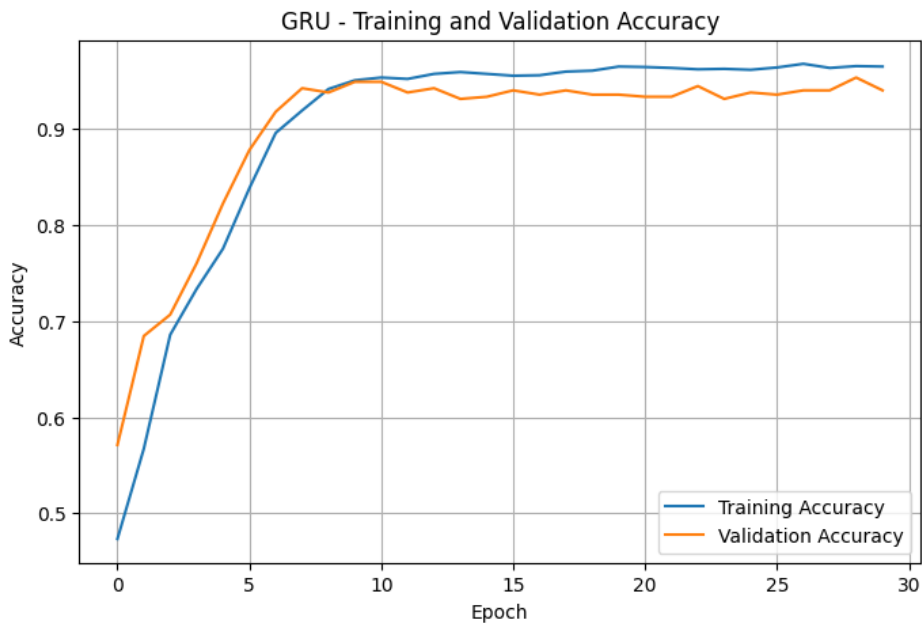


Figure 7: GRU Training and Validation Accuracy

Inference:

The GRU accuracy increases rapidly and reaches more than 90% by around the ninth epoch. After this, training accuracy stays near 96%, while validation accuracy stays around 94–96%. The curves are close for most of the later epochs, so the GRU shows good convergence and only a small generalization gap.

14. Performance Evaluation

The three models were evaluated on the independent test set. Macro averaging was used for Precision, Recall and F1-score.

Metric	RNN	LSTM	GRU
Accuracy (%)	74.8889	94.8889	94.4444
Macro Precision (%)	75.2202	94.8425	94.3957
Macro Recall (%)	74.8889	94.8889	94.4444
Macro F1 (%)	75.0342	94.8588	94.4122
Parameters	1974	6006	4758
Training Time (s)	27.4841	23.6668	21.1236

The RNN obtained 74.89% test accuracy, while the LSTM and GRU obtained 94.89% and 94.44%, respectively. The LSTM also had the highest test F1-score of 94.86%. The GRU used fewer parameters than the LSTM and also had the lowest training time among the three models in this run.

15. Confusion Matrix Analysis

RNN Confusion Matrix

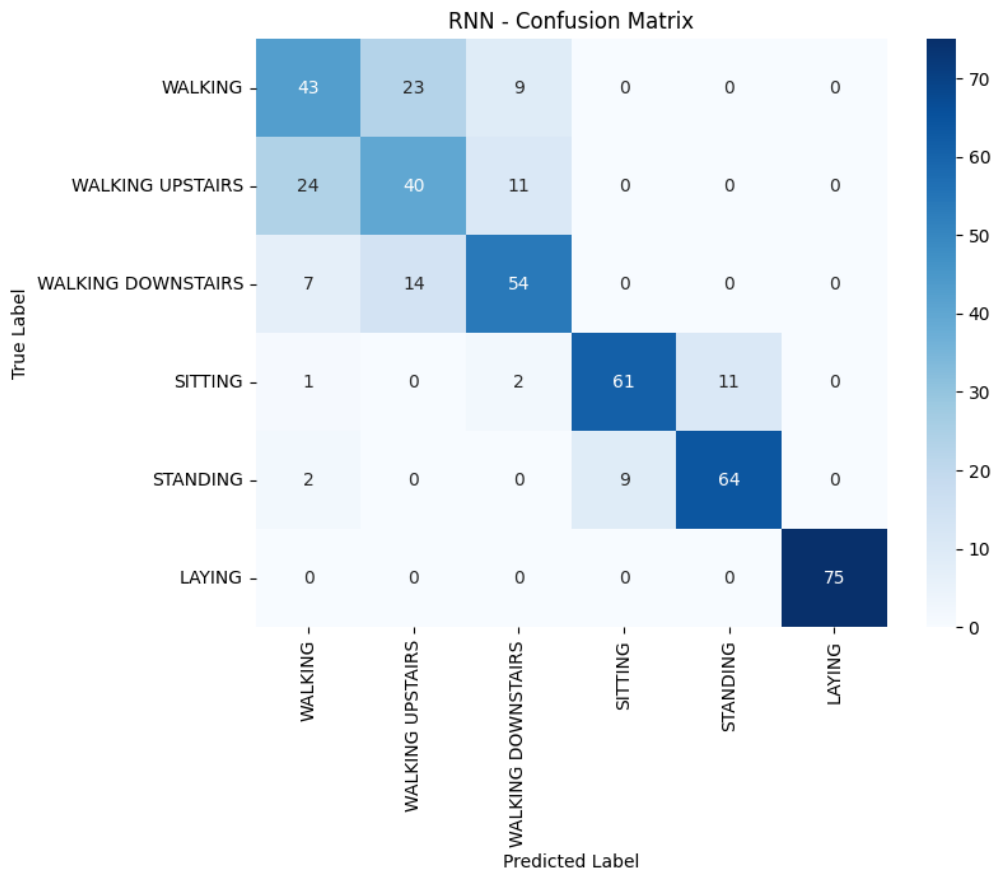


Figure 8: RNN Confusion Matrix

For the RNN, LAYING had the highest recognition rate with 75 out of 75 samples classified correctly. WALKING UPSTAIRS had the largest number of errors, with 35 samples misclassified. The main confusion was between WALKING UPSTAIRS and WALKING, where 24 WALKING UPSTAIRS samples were predicted as WALKING.

LSTM Confusion Matrix

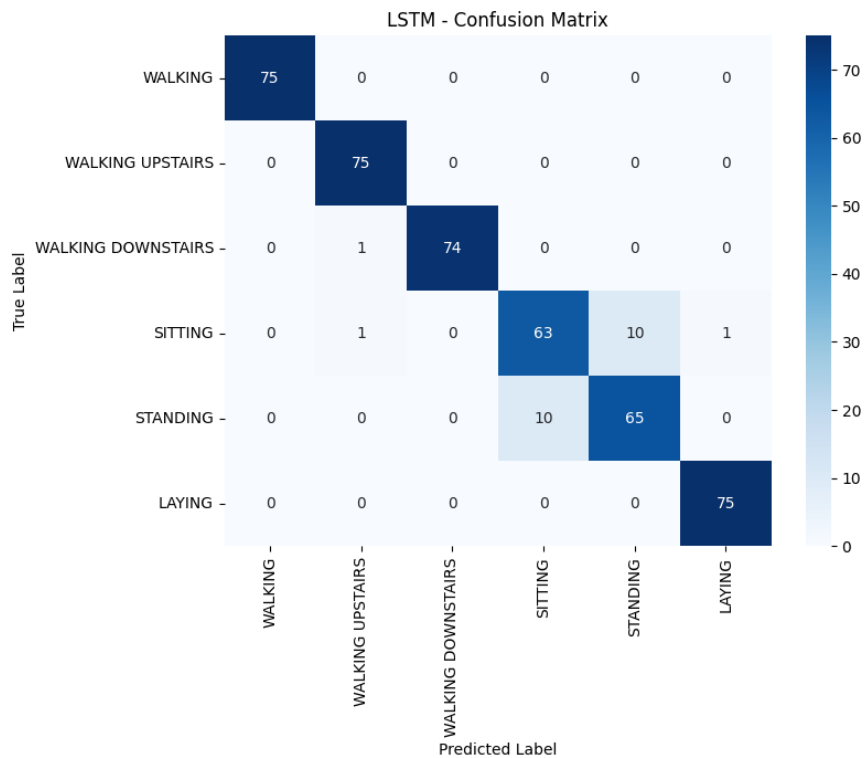


Figure 9: LSTM Confusion Matrix

For the LSTM, WALKING had 75 out of 75 correct predictions and LAYING was also classified correctly for all 75 samples. SITTING had the largest number of errors, mainly because 10 SITTING samples were predicted as STANDING. The reverse confusion also happened, with 10 STANDING samples predicted as SITTING.

GRU Confusion Matrix

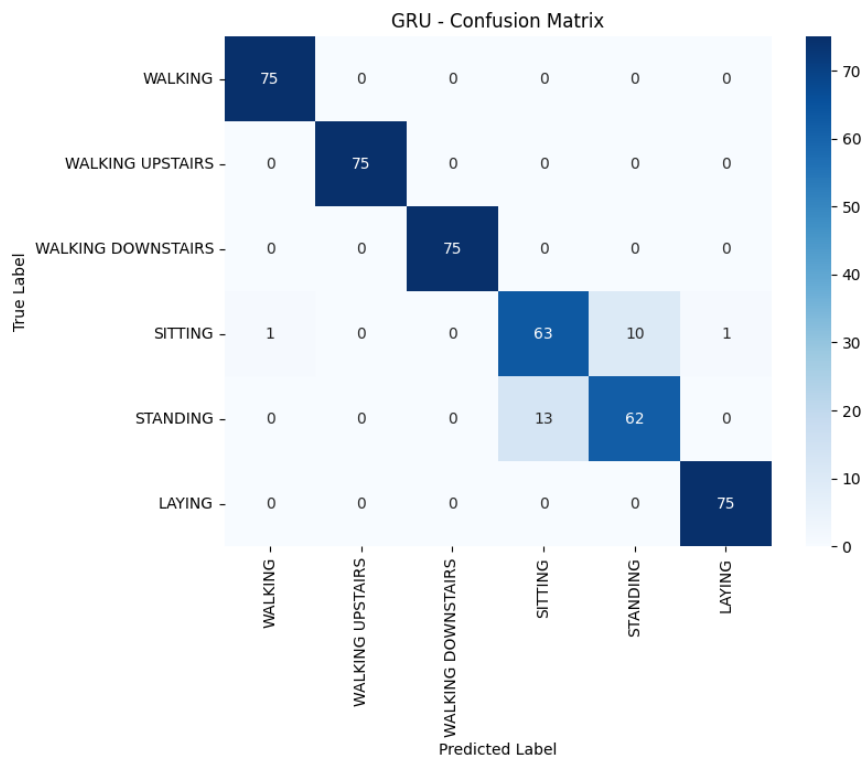


Figure 10: GRU Confusion Matrix

For the GRU, WALKING, WALKING UPSTAIRS, WALKING DOWNSTAIRS and LAYING were all classified correctly for all 75 test samples. The largest error was for STANDING, where 13 samples were predicted as SITTING. The same SITTING–STANDING confusion was also seen in the LSTM, so this error was consistent between these two models.

Overall inference:

The RNN has much more confusion among the three walking activities, especially WALKING and WALKING UPSTAIRS. The LSTM and GRU almost completely remove these walking-related errors. Their remaining errors are mainly between SITTING and STANDING, which have more similar sensor patterns in the selected data.

16. RNN vs. LSTM vs. GRU Comparison

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1974	6006	4758
Training time (s)	27.4841	23.6668	21.1236
Test F1-score (%)	75.0342	94.8588	94.4122

Plot 5: Model Performance Comparison

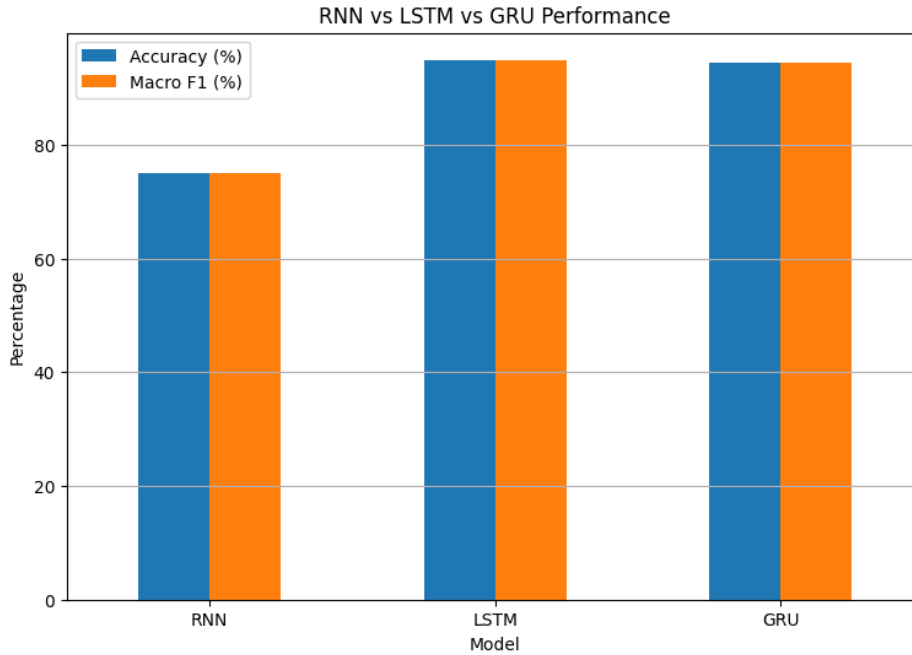


Figure 11: RNN, LSTM and GRU Test Performance Comparison

Inference:

The RNN has the smallest number of parameters, 1974, but its test F1 is only 75.03%. The LSTM and GRU have more parameters but their F1 scores are much higher at 94.86% and 94.41%. In this actual run, the GRU had fewer parameters than the LSTM and also the lowest training time, while giving almost the same F1-score. Therefore, in this experiment, the RNN has lower model complexity but also much lower predictive performance. LSTM and GRU give much higher accuracy and F1-scores with more parameters. Between the two gated models, the GRU uses fewer parameters and has a lower training time while giving a Macro F1-score close to the LSTM. This shows the trade-off between predictive performance, model complexity, and training cost for the three models.

17. Effect of Sequence Length

The models were tested using sequence lengths of 32, 64 and 128.

T	Model	Accuracy (%)	F1 (%)	Parameters	Time (s)
32	RNN	88.2222	88.1343	1974	17.5913
32	LSTM	95.5556	95.5377	6006	16.8461
32	GRU	93.3333	93.3505	4758	16.4648
64	RNN	86.0000	86.0327	1974	20.0871
64	LSTM	94.4444	94.4260	6006	18.3420
64	GRU	94.4444	94.4280	4758	18.4889
128	RNN	70.2222	69.9462	1974	26.6462
128	LSTM	95.7778	95.7776	6006	21.4141
128	GRU	94.8889	94.8720	4758	21.2254

Plot 6: Sequence Length vs. Test F1-score

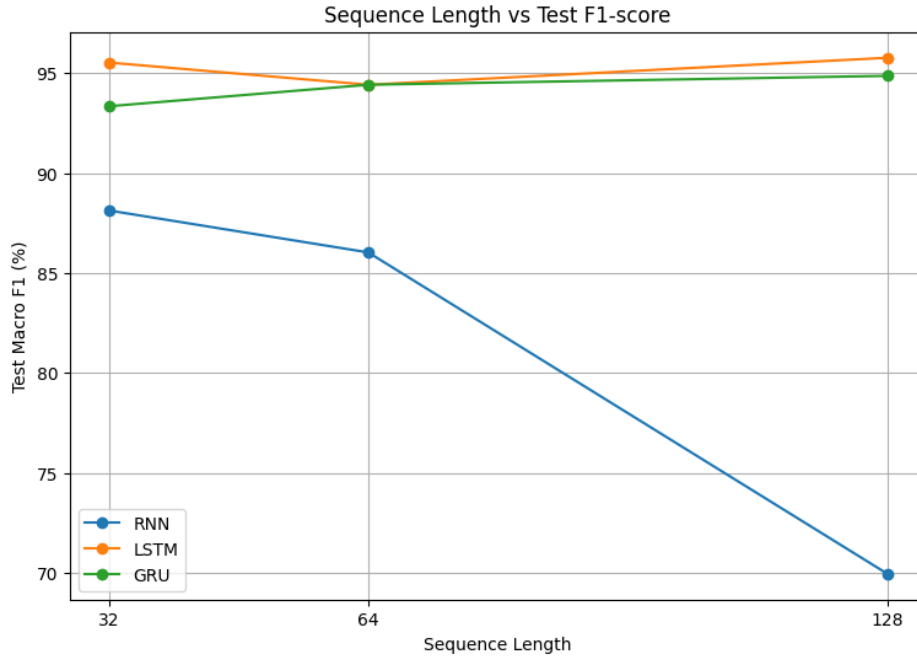


Figure 12: Sequence Length vs. Test F1-score

Inference:

The RNN performance decreases as the sequence becomes longer. Its F1 falls from 88.13% at $T = 32$ to only 69.95% at $T = 128$, while its training time also increases from 17.59 s to 26.65 s. The LSTM and GRU do not show the same drop; the LSTM reaches 95.78% F1 at $T = 128$ and the GRU reaches 94.87%. This shows that increasing temporal context does not always improve classification. The simple RNN had more difficulty retaining useful information over the longer sequences, whereas the LSTM and GRU use gating mechanisms to retain relevant information and handle longer temporal dependencies better in this experiment.

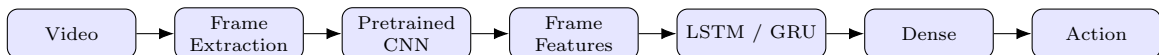
18. Video Understanding Using CNN and RNN

This part demonstrates the application:

Video Understanding using CNN + RNN

A video is a sequence of image frames. CNNs are used to extract spatial features from individual frames, while RNN/LSTM/GRU models the temporal relationship among those features.

The complete pipeline is:



Recommended Dataset

Use a small subset of the UCF101 action-recognition dataset:

<https://www.crcv.ucf.edu/data/UCF101.php>

Select only 3–5 action classes and a small number of videos per class so that the experiment remains computationally manageable.

Possible classes include:

- Basketball
- Biking
- Walking
- Running
- TennisSwing

The objective is to understand the CNN–recurrent pipeline rather than to train a state-of-the-art video recognition system.

19. Expected Video Input and Output

Sample 10 frames uniformly from each video.

Each frame is resized to

$$224 \times 224 \times 3.$$

A pretrained CNN such as MobileNetV2 is used only as a feature extractor.

If the CNN produces a feature vector of dimension D , then the recurrent network receives:

$$X \in \mathbb{R}^{10 \times D}.$$

For a batch of B videos:

$$X \in \mathbb{R}^{B \times 10 \times D}.$$

The expected output is a probability vector over the selected action classes.

For example:

Predicted class : Basketball
Actual class : Basketball
Confidence : 0.xx

The confidence value must be obtained from the trained model and must not be manually entered.

20. CNN Feature Extraction

MobileNetV2 was used as a frozen feature extractor. Its total parameter count was 2,257,984 and its trainable parameter count was 0.



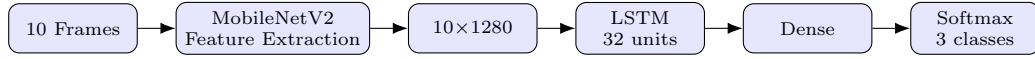
The CNN was kept frozen and was not trained again on the video data. The extracted features were then arranged as a sequence of 10 feature vectors and given to the LSTM.

Observation:

The CNN converts every 224×224 RGB frame into a 1280-dimensional feature vector. For 10 frames, the extracted features have the shape (10, 1280) and are supplied to the LSTM as a sequence. Thus, the LSTM receives 10 feature vectors instead of the original image pixels.

21. CNN–LSTM / CNN–GRU Model

The main video model used a frozen MobileNetV2 followed by an LSTM with 32 units and a final softmax layer.



Plot 7: Video Sample Frames

A representative sequence of 10 uniformly sampled frames was selected from a video.

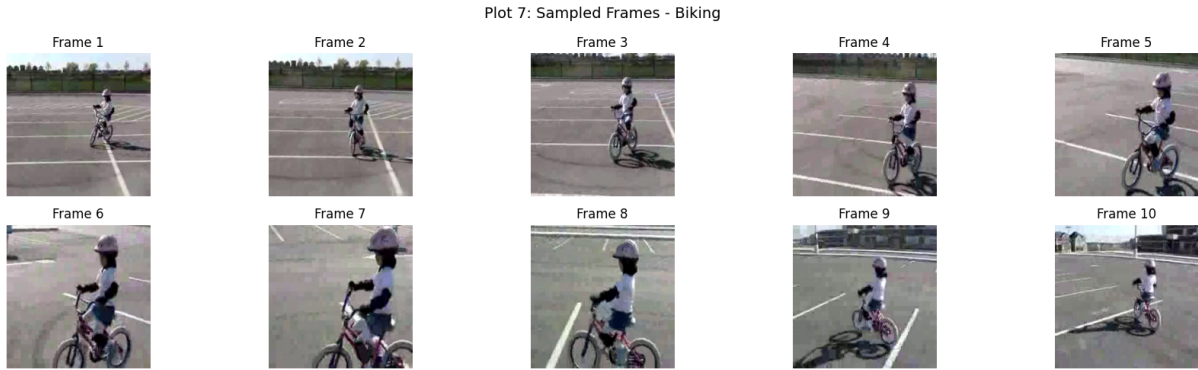


Figure 13: Sampled Frames from a Video

Inference:

The sampled frames show the visual content of the action changing across time. The CNN captures spatial information such as the objects, people, and visual patterns present in each frame, while the LSTM learns how these frame-level features change across the sequence to identify the action.

Plot 8: Video Training and Validation Curves

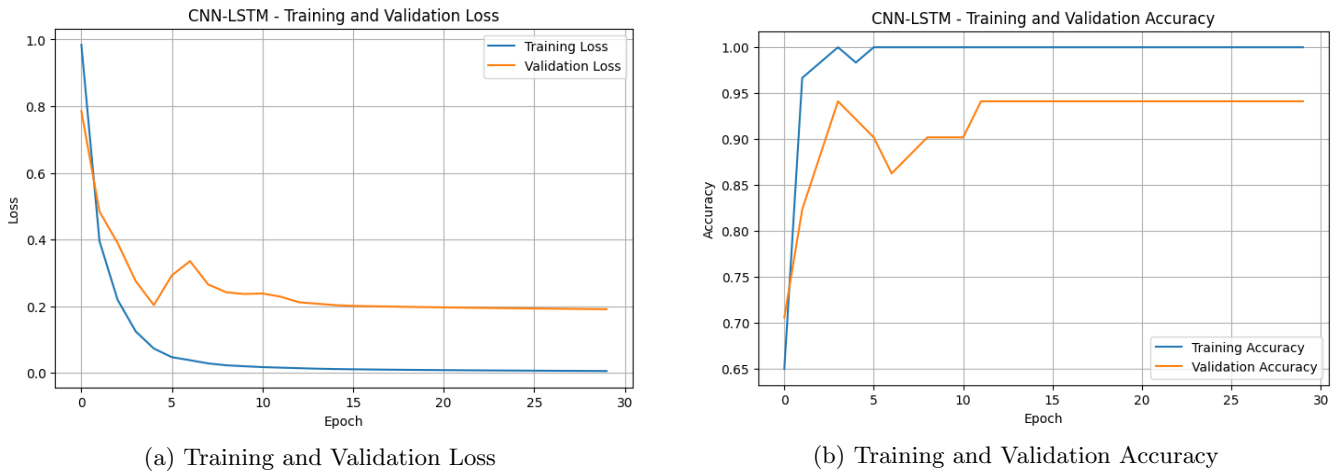


Figure 14: CNN-LSTM Training and Validation Curves

Inference:

The CNN-LSTM training accuracy reaches 100% quickly, while the validation accuracy stays around 94%. The training loss becomes almost zero, while the validation loss remains around 0.19. This shows a training-validation gap and some overfitting, although the validation accuracy remains high.

Plot 9: Video Confusion Matrix

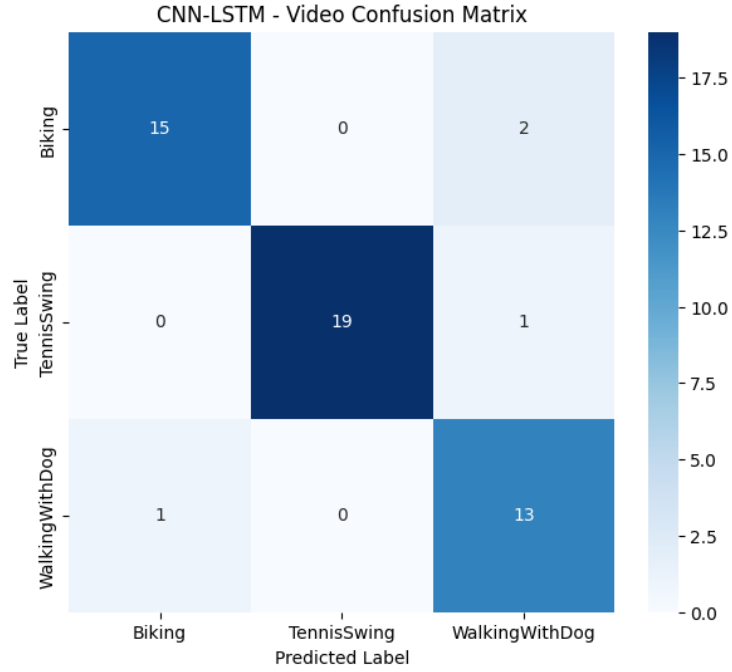


Figure 15: CNN-LSTM Video Confusion Matrix

Inference:

TennisSwing had 19 correct predictions out of 20 test samples. Biking had 15 correct predictions out of 17, while WalkingWithDog had 13 correct predictions out of 14. The main errors were Biking predicted as WalkingWithDog and one WalkingWithDog sample predicted as Biking. This shows that the model separated TennisSwing well, while a few confusions remained between the other two actions.

Video Model Results:

Metric	CNN-LSTM
Accuracy (%)	92.1569
Macro Precision (%)	91.6667
Macro Recall (%)	92.0308
Macro F1 (%)	91.6706
Parameters	168163
Training Time (s)	6.0727
CNN Feature Dimension	1280

22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another.

For a simple laboratory task, construct a synthetic reversal dataset.

Example:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range.

The task is intentionally simple so that the encoder-decoder mechanism can be studied without a large external dataset.



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

23. Sequence-to-Sequence Input and Output

Five test examples obtained from the trained Seq2Seq model are shown below.

Sample	Input Sequence	Predicted Output	Actual Output
1	[2,3,4,7]	[7,4,3,2]	[7,4,3,2]
2	[5,2,3,6]	[6,3,2,5]	[6,3,2,5]
3	[8,4,8,1]	[1,8,4,8]	[1,8,4,8]
4	[1,5,2,4]	[4,2,5,1]	[4,2,5,1]
5	[3,6,3,2]	[2,3,6,3]	[2,3,6,3]

All five displayed test sequences were reversed correctly by the model.

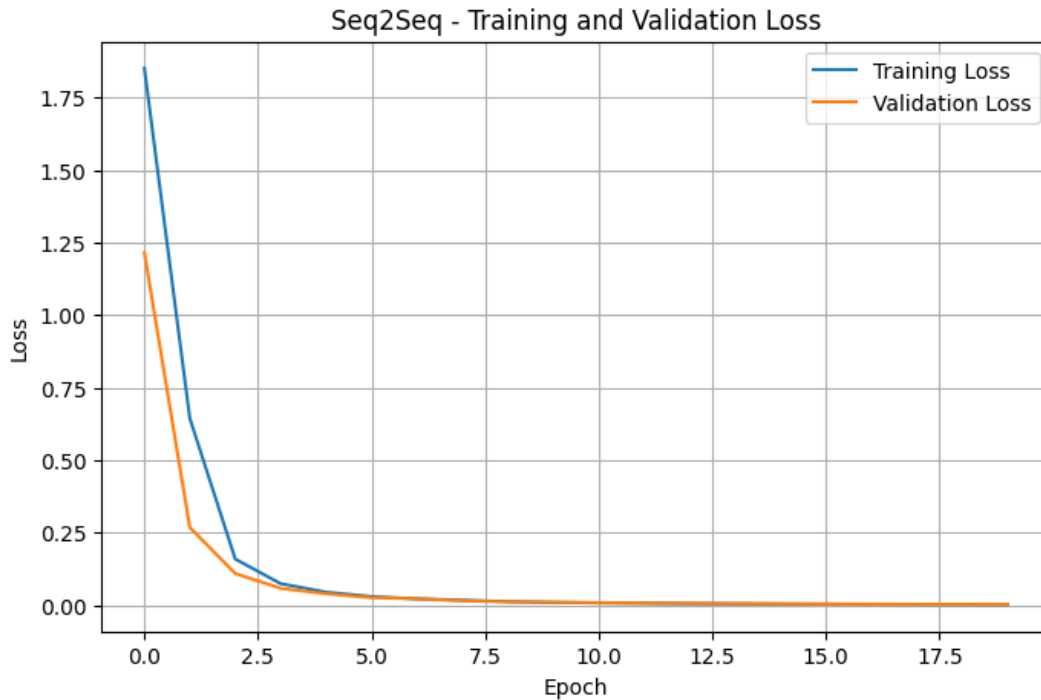


Figure 16: Seq2Seq Training and Validation Loss

24. Sequence-to-Sequence Evaluation

The final Seq2Seq results were:

Metric	Result
Token Accuracy	100.0%
Sequence Accuracy	100.0%
Final Training Loss	0.0023
Final Validation Loss	0.0028

The training loss decreased from 1.8512 in the first epoch to 0.0023 in the final epoch. The validation loss decreased from 1.2158 to 0.0028. Both training and validation accuracy reached 100%.

Inference:

In this experiment, token accuracy and sequence accuracy are both 100% because every predicted token was correct and all the complete test sequences were also correct. Normally, sequence accuracy can be lower because one wrong token is enough to make the whole sequence incorrect. This difference was not visible in our final result because the model correctly predicted all the tested sequences.

25. Consolidated Results

Model	Accuracy	Precision	Recall	F1	Parameters
RNN	74.8889	75.2202	74.8889	75.0342	1974
LSTM	94.8889	94.8425	94.8889	94.8588	6006
GRU	94.4444	94.3957	94.4444	94.4122	4758
CNN-LSTM	92.1569	91.6667	92.0308	91.6706	168163

Sequence-to-Sequence Results

Metric	Result
Input sequences	5000
Input length	4
Output length	4
Token Accuracy	100.0%
Sequence Accuracy	100.0%
Final Training Loss	0.0023
Final Validation Loss	0.0028
Parameters	68171

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn from the experiment?

The following inferences are mandatory:

- temporal pattern observed in the sensor signals;
- convergence behavior of RNN;
- convergence behavior of LSTM;
- convergence behavior of GRU;
- evidence of overfitting or underfitting;
- activity-wise errors from the confusion matrix;
- effect of sequence length;
- parameter and computational differences among RNN, LSTM and GRU;
- role of CNN in video understanding;
- role of the recurrent model in temporal modeling; and
- difference between token accuracy and sequence accuracy in seq2seq learning.

27. Discussion Questions

1. What is a recurrent neural network?

A recurrent neural network is a neural network used for sequence data. It processes the input one time step at a time and keeps a hidden state to carry information from previous time steps. This allows the network to use information from earlier inputs when processing later inputs.

2. What information is represented by the hidden state?

The hidden state stores information learned from the current input and the previous time steps. In this experiment, it helps the model use the sensor measurements across the sequence. Therefore, the hidden state acts as a summary of relevant information seen so far in the sequence.

3. Why is sequence order important?

The order tells the model how the sensor values change with time. For example, the 128 measurements in the HAR data are not independent values. Their order helps distinguish different activities because different activities produce different temporal patterns.

4. Explain the difference between a feed-forward network and an RNN.

A feed-forward network processes an input without keeping information from previous inputs. An RNN has a hidden state, so information from earlier time steps can be used while processing later time steps. This makes RNNs suitable for data where the relationship between inputs depends on their order.

5. **Explain Backpropagation Through Time.**
BPTT is the method used to train an RNN by sending the error backwards through the different time steps of the sequence. The recurrent weights are updated using the errors from these time steps. In this way, the model learns how its earlier hidden states affected the final output.
6. **Why can gradients vanish during BPTT?**
During BPTT, gradients are repeatedly multiplied through many time steps. If these values become very small, the gradient can approach zero and earlier time steps receive very small updates.
7. **Why can gradients explode during BPTT?**
If the repeated multiplications produce very large values, the gradient can become extremely large. This is called the exploding gradient problem.
8. **What are long-term dependencies?**
Long-term dependencies are relationships where information from an earlier part of a sequence is needed much later. LSTM and GRU are designed to handle such information better than a simple RNN.
9. **What is the role of the LSTM cell state?**
The cell state carries information through the sequence. The gates control what information should be kept, added or removed.
10. **What are the forget, input and output gates in LSTM?**
The forget gate decides what old information should be removed. The input gate controls new information added to the cell state. The output gate controls what information is passed to the hidden state.
11. **What are the reset and update gates in GRU?**
The reset gate controls how much previous information is used when forming a new state. The update gate controls how much old and new information is kept.
12. **Compare LSTM and GRU.**
Both use gates to handle sequence information. In this experiment, LSTM had 6006 parameters and a test F1 of 94.86%, while GRU had 4758 parameters and a test F1 of 94.41%. Their results were close, but GRU used fewer parameters.
13. **Why does GRU generally use a simpler gating mechanism than LSTM?**
GRU uses two main gates, reset and update, and does not use a separate cell state. LSTM uses more gates and has both a hidden state and cell state.
14. **Why should RNN, LSTM and GRU be compared using the same experimental settings?**
Using the same data, batch size, optimizer and evaluation method makes the comparison fair. In this experiment all three used the same classification setup.
15. **What does the confusion matrix reveal that accuracy alone does not?**
The confusion matrix shows which classes are being confused. For example, the RNN had a large confusion between WALKING UPSTAIRS and WALKING, which cannot be seen from accuracy alone.
16. **Why should macro F1-score be reported for multi-class classification?**
Macro F1 calculates the F1-score for each class and then gives equal importance to all classes. It therefore shows how the model performs across the different activity classes.
17. **What is the significance of sequence length?**
Sequence length decides how much temporal information is given to the model. In our experiment, increasing the sequence length affected the models differently. The RNN dropped strongly at 128 steps, while LSTM and GRU stayed close to their earlier performance.
18. **Why can a CNN be used as a feature extractor for video?**
A CNN can convert an image frame into useful numerical features. This avoids giving all the raw image pixels directly to the recurrent network.
19. **What type of information is extracted by the CNN?**
The CNN extracts spatial information from each frame, such as useful visual patterns present in the image.
20. **What type of information is learned by the recurrent network?**
The recurrent network learns the temporal relationship between the features of different frames.
21. **Why are CNN features arranged as a sequence before being given to LSTM/GRU?**
A video contains multiple frames in time order. Giving the CNN features as a sequence allows the recurrent network to learn how the visual information changes from one frame to another.
22. **Explain the encoder and decoder in sequence-to-sequence learning.**
The encoder reads the input sequence and converts it into a learned state. The decoder uses this state to generate the output sequence one token at a time.
23. **What is teacher forcing?**
Teacher forcing means giving the correct previous output token to the decoder while training it to predict the next token. This was used when preparing the decoder inputs in the Seq2Seq experiment.
24. **Differentiate token accuracy and sequence accuracy.**
Token accuracy checks individual predicted tokens. Sequence accuracy checks whether the complete sequence is correct. In our experiment both were 100% because the complete test sequences were predicted correctly.
25. **Why must the test set remain untouched during model selection?**

The test set should represent unseen data. If it is repeatedly used while changing the model, the test result can influence the model selection and will no longer give a clean final evaluation.

28. Additional Exercises

Exercise 1: Effect of Recurrent Units

The recurrent units were changed from 32 to 16 and 64.

Model	Units	Accuracy (%)	F1 (%)	Parameters	Time (s)
RNN	16	71.1111	70.9744	790	26.1441
RNN	64	83.1111	83.1442	5878	27.4527
LSTM	16	94.0000	93.9313	2038	20.9821
LSTM	64	96.0000	96.0000	20086	21.7952
GRU	16	94.4444	94.4107	1670	21.6705
GRU	64	94.8889	94.8588	15542	21.0888

Inference:

Increasing the units improved the RNN accuracy from 71.11% to 83.11% and its F1 from 70.97% to 83.14%. The LSTM also increased from 94.00% to 96.00% accuracy. The GRU changed only slightly from 94.44% to 94.89%, while the parameter count increased a lot for all three models.

Exercise 2: GRU and LSTM with the Same Units

Model	Units	Accuracy (%)	F1 (%)	Parameters
LSTM	32	94.8889	94.8588	6006
GRU	32	94.4444	94.4122	4758

The LSTM had a slightly higher F1-score than GRU, but GRU used 1248 fewer parameters. As both models use the same units 32 units, the difference mainly comes from their different gating structures. Also, in this experiment, the GRU achieved almost similar performance with fewer parameters itself.

Exercise 3: Second Recurrent Layer

A second LSTM layer was added.

Model	Accuracy (%)	F1 (%)	Parameters	Time (s)
Single LSTM	94.8889	94.8588	6006	23.6668
Two-layer LSTM	93.7778	93.7595	14326	31.1477

Inference:

The second layer of LSTM did not improve the test result much in this run. Accuracy has decreased slightly from 94.89% to 93.78% and F1 also slightly decreased from 94.86% to 93.76%. But at the same time, parameter count and training time increased, so extra layer added more cost without improving the measured test performance much here. The additional complexity did not give us useful improvement for this dataset.

Exercise 4: Bidirectional LSTM

Model	Accuracy (%)	F1 (%)	Parameters	Time (s)
LSTM	94.8889	94.8588	6006	23.6668
Bidirectional LSTM	92.6667	92.6657	11894	30.9918

Inference: The bidirectional LSTM did not improve the result on this dataset. Its F1-score was 92.67%, compared with 94.86% for the normal LSTM. It also had almost twice the parameters and took longer to train. Therefore using information from both directions also did not provide an advantage for this classification task in this experiment.

Exercise 5: Sequence Length and Computational Cost

The training times increased as the sequence length increased for the three main models.

For RNN, the time increased from 17.59 s at $T = 32$ to 26.65 s at $T = 128$. For LSTM it increased from 16.85 s to 21.41 s, and for GRU it increased from 16.46 s to 21.23 s. Therefore, longer sequences required more training time in all three models.

Exercise 6: CNN-LSTM vs. CNN-GRU

The same extracted CNN features were used for both recurrent models.

Model	Accuracy (%)	F1 (%)	Parameters	Time (s)
CNN-LSTM	92.1569	91.6706	168163	6.0727
CNN-GRU	92.1569	92.0655	126243	5.8784

Inference:

Both video models also obtained the same test accuracy of 92.16 only%. The CNN-GRU had slightly higher F1-score of 92.07% when compared with 91.67% for CNN-LSTM. It also used fewer parameters and had a slightly lower measured training time, while both models used same CNN features. Since both models used the same CNN features, the small difference came from the recurrent part of the model, with the GRU giving similar performance at lower computational cost in this experiment.

Exercise 7: Different Input and Output Lengths

The input sequence had length 4 and the output sequence had length 3.

Input shape = (5000, 4)

Output shape = (5000, 3)

Example:

$[7, 4, 8, 5] \rightarrow [5, 8, 4]$.

At the final epoch, training accuracy and validation accuracy were both 100%. The final training loss was 0.0015 and validation loss was 0.0016.

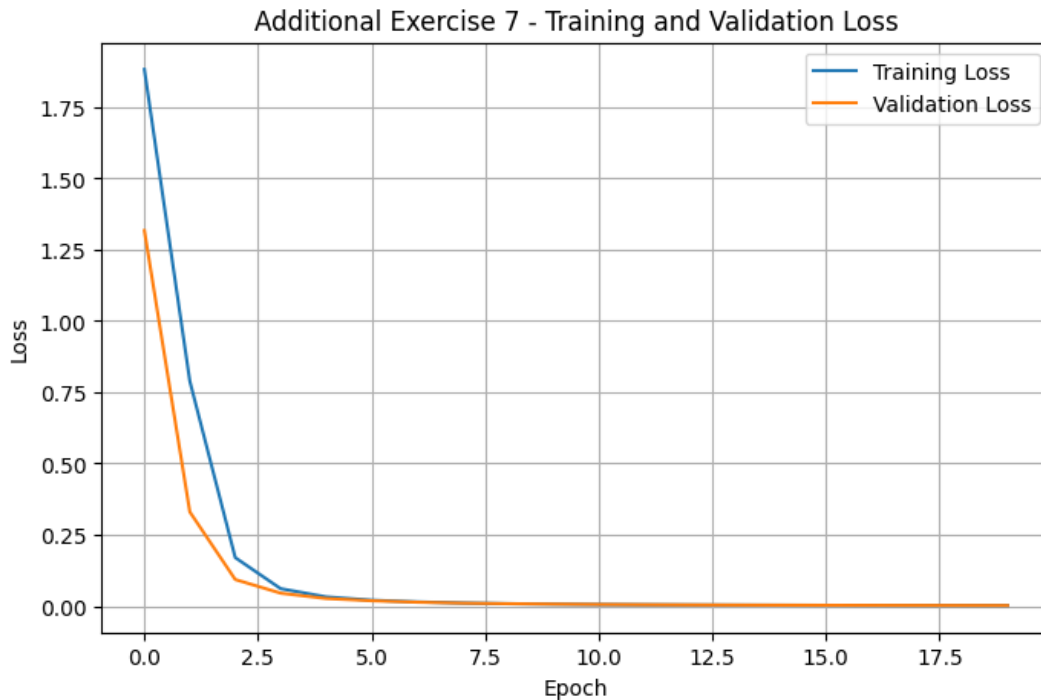


Figure 17: Seq2Seq with Different Input and Output Lengths

Inference:

The modified model has successfully learned the 4 to 3 mapping correctly. The loss has decreased to 0.0015 for training and 0.0016 for validation, while both accuracies reached 100%. Therefore, changing the output length did not stop the encoder-decoder model from learning the required reversal-based mapping.

29. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete sequence-learning pipeline:

Sequential Data → Preprocessing → RNN/LSTM/GRU → Evaluation

and the video-understanding pipeline:

Video → Frames → CNN Features → LSTM/GRU → Action Prediction

They should also understand the encoder–decoder formulation:

Input Sequence → Encoder → Context → Decoder → Output Sequence

The final report should demonstrate both implementation and interpretation. Numerical performance values must be obtained from the student’s own execution and should not be assumed in advance.

30. Source Code

The complete source code, including preprocessing, model implementation, training, evaluation, plots, and additional exercises, is available in the following GitHub repository:

https://github.com/srinithi0406/Deep_Learning/tree/main/DL_6

31. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>